

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <time.h>
6
7 #define NUM_ITEMS 10
8
9 int main()
10 {
11     int pipefd[2];
12     pid_t pid;
13     int data;
14     struct timespec start, end;
15
16     // Create pipe
17     if (pipe(pipefd) == -1)
18     {
19         perror("pipe");
20         exit(1);
21     }
22
23     // Start measuring time
24     clock_gettime(CLOCK_MONOTONIC, &start);
25
26     pid = fork();
27
28     if (pid < 0)
29     {
30         perror("fork");
31         exit(1);
32     }
33
34     if (pid > 0)
35     {
36         // ===== PRODUCER (PARENT) =====
37
38         close(pipefd[1]); // Close unused write end
39         printf("Producer (Parent) started. PID = %d\n", getpid());
40
41         for (int i = 1; i <= NUM_ITEMS; i++)
42         {
43             data = i * 10;
44
45             printf("Producer: Sending %d\n", data);
46             write(pipefd[1], &data, sizeof(data));
47             usleep(100000); // 0.1 second
48         }
49         close(pipefd[1]); // Signal end of data
50
51         wait(NULL); // Wait for consumer
52
53         clock_gettime(CLOCK_MONOTONIC, &end);
54
55         double elapsed =
56             (end.tv_sec - start.tv_sec) +
57             (end.tv_nsec - start.tv_nsec) / 1e9;
58
59         printf("\nCommunication completed.\n");
60         printf("Time taken: %.6f seconds\n", elapsed);
61         printf("Items transferred: %d\n", NUM_ITEMS);
62         printf("Communication rate: %.2f items/second\n",
63             NUM_ITEMS / elapsed);
64     }
65     else
66     {
67         // ===== CONSUMER (CHILD) =====
68
69         close(pipefd[1]); // Close unused write end
70
71         printf("Consumer (Child) started. PID = %d\n", getpid());
72
73         while (read(pipefd[0], &data, sizeof(data)) > 0)
74         {
75             printf("Consumer: Received %d\n", data);
76         }
77
78         close(pipefd[0]);
79
80         printf("Consumer finished.\n");
81
82         exit(0);
83     }
84
85     return 0;

```

```

kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ gedit practical5.c

** (gedit:16513): WARNING **: 03:42:58.685: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:16513): WARNING **: 03:42:58.686: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ gcc practical5.c -o practical5
kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ ./practical5
Producer (Parent) started. PID = 21454
Producer: Sending 10
Consumer (Child) started. PID = 21455
Consumer: Received 10
Producer: Sending 20
Consumer: Received 20
Producer: Sending 30
Consumer: Received 30
Producer: Sending 40
Consumer: Received 40
Producer: Sending 50
Consumer: Received 50
Producer: Sending 60
Consumer: Received 60
Producer: Sending 70
Consumer: Received 70
Producer: Sending 80
Consumer: Received 80
Producer: Sending 90
Consumer: Received 90
Producer: Sending 100
Consumer: Received 100
Consumer finished.

Communication completed.
Time taken: 1.005917 seconds
Items transferred: 10
Communication rate: 9.94 items/second

```

Implementing `ls -l | grep ".c"`

```

kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ ls -l | grep ".c"
-rwxrwxrwx 1 kaushik kaushik 255 Aug 22 02:51 permission.c
-rwxrwxrwx 1 kaushik kaushik 1420 Aug 22 03:49 pipe_grep.c
-rwxrwxrwx 1 kaushik kaushik 16392 Aug 5 03:49 practical
-rwxrwxrwx 1 kaushik kaushik 16392 Aug 5 03:50 practical1
-rwxrwxrwx 1 kaushik kaushik 787 Aug 5 03:49 practical1.c
-rwxrwxrwx 1 kaushik kaushik 16176 Aug 5 04:00 practical2
-rwxrwxrwx 1 kaushik kaushik 755 Aug 5 04:00 practical2.c
-rwxrwxrwx 1 kaushik kaushik 16296 Aug 8 03:07 practical3
-rwxrwxrwx 1 kaushik kaushik 1257 Aug 8 03:06 practical3.c
-rwxrwxrwx 1 kaushik kaushik 16344 Aug 22 03:24 practical4
-rwxrwxrwx 1 kaushik kaushik 1089 Aug 22 03:23 practical4.c
-rwxrwxrwx 1 kaushik kaushik 16520 Aug 22 03:48 practical5
-rwxrwxrwx 1 kaushik kaushik 1902 Aug 22 03:48 practical5.c
-rwxrwxrwx 1 kaushik kaushik 22 Aug 5 04:01 source.txt
-rwxrwxrwx 1 kaushik kaushik 1090 Aug 22 03:27 waitpid_demo.c
-rwxrwxrwx 1 kaushik kaushik 837 Aug 22 03:37 zombie.c

```

Open ▾ 

pipe_grep.c
/mnt/c/WINDOWS/system32/os/practical

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main()
7 {
8     int pipefd[2];
9     pid_t pid1, pid2;
10
11     // Create pipe
12     if (pipe(pipefd) == -1)
13     {
14         perror("pipe");
15         exit(1);
16     }
17
18     // Create first child for ls -l
19     pid1 = fork();
20
21     if (pid1 < 0)
22     {
23         perror("fork");
24         exit(1);
25     }
26
27     if (pid1 == 0)
28     {
29         // Child 1: execute ls -l
30         close(pipefd[0]); // Close read end
31
32         // Redirect stdout to pipe
33         dup2(pipefd[1], STDOUT_FILENO);
34
35         close(pipefd[1]);
36
37         execlp("ls", "ls", "-l", NULL);
38
39         // Only reached if exec fails
40         perror("execlp ls");
41         exit(1);
42     }
43
44     // Create second child for grep
45     pid2 = fork();
46
47     if (pid2 < 0)
48     {
49         perror("fork");
50         exit(1);
51     }
52
53     if (pid2 == 0)
54     {
55         // Child 2: execute grep ".c"
56
57         close(pipefd[1]); // Close write end
58
59         // Redirect stdin from pipe
60         dup2(pipefd[0], STDIN_FILENO);
61
62         close(pipefd[0]);
63
64         execlp("grep", "grep", ".c", NULL);
65
66         // Only reached if exec fails
67         perror("execlp grep");
68         exit(1);
69     }
70
71     // Parent closes both pipe ends
72     close(pipefd[0]);
73     close(pipefd[1]);
74
75     // Wait for both children
76     waitpid(pid1, NULL, 0);
77     waitpid(pid2, NULL, 0);
78
79     printf("Command execution completed.\n");
80
81     return 0;
82 }
```

```

kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ gedit pipe_grep.c

** (gedit:26707): WARNING **: 03:52:03.676: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:26707): WARNING **: 03:52:03.676: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ gcc pipe_grep.c -o pipe_grep
kaushik@LAPTOP-ASUSTUF3107 :/mnt/c/WINDOWS/system32/os/practical $ ./pipe_grep
-rwxrwxrwx 1 kaushik kaushik 255 Aug 22 02:51 permission.c
-rwxrwxrwx 1 kaushik kaushik 1420 Aug 22 03:49 pipe_grep.c
-rwxrwxrwx 1 kaushik kaushik 16392 Aug 5 03:49 practical
-rwxrwxrwx 1 kaushik kaushik 16392 Aug 5 03:50 practical1
-rwxrwxrwx 1 kaushik kaushik 787 Aug 5 03:49 practical1.c
-rwxrwxrwx 1 kaushik kaushik 16176 Aug 5 04:00 practical2
-rwxrwxrwx 1 kaushik kaushik 755 Aug 5 04:00 practical2.c
-rwxrwxrwx 1 kaushik kaushik 16296 Aug 8 03:07 practical3
-rwxrwxrwx 1 kaushik kaushik 1257 Aug 8 03:06 practical3.c
-rwxrwxrwx 1 kaushik kaushik 16344 Aug 22 03:24 practical4
-rwxrwxrwx 1 kaushik kaushik 1089 Aug 22 03:23 practical4.c
-rwxrwxrwx 1 kaushik kaushik 16520 Aug 22 03:48 practical5
-rwxrwxrwx 1 kaushik kaushik 1902 Aug 22 03:48 practical5.c
-rwxrwxrwx 1 kaushik kaushik 22 Aug 5 04:01 source.txt
-rwxrwxrwx 1 kaushik kaushik 1090 Aug 22 03:27 waitpid demo.c
-rwxrwxrwx 1 kaushik kaushik 837 Aug 22 03:37 zombie.c
Command execution completed.

```